

Practical 7

AIM: Implementation of making a change problem using dynamic programming

1 ALGORITHM:

Algorithm
Input coins denominations coins C.
 number of denomination = m
 Target amount = V.
 output = minimum number of coins to make V.
Steps
 Initialize an array dp[0...V] where
 Target amount = V.
 output = ~~minimum number of coins to make V.~~
 * Set dp[0] = 0 (no coins needed for amount 0).
 for all other i, set dp[i] = infinity.
 For each amount i to V:
 for each coins[j].
 if coins[j] <= i then
 dp[i] = min(dp[i - coins[j]] + 1).
 After filling the table dp[V] gives minimum coins.
 if dp[V] = infinity, no solution exists.

2 PROGRAM:

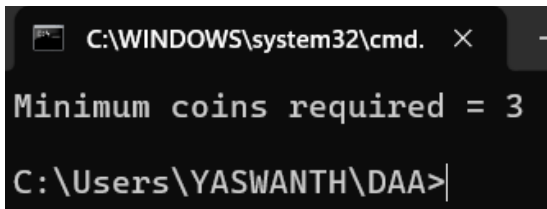
```
#include <iostream>
#include <climits>
using namespace std;
int minCoins(int coins[], int m, int V) {
    int dp[V+1];
    dp[0] = 0;
    for (int i = 1; i <= V; i++)
        dp[i] = INT_MAX;
    for (int i = 1; i <= V; i++) {
        for (int j = 0; j < m; j++) {
            if (coins[j] <= i) {
                int sub_res = dp[i - coins[j]];
                if (sub_res != INT_MAX && sub_res + 1 < dp[i])
                    dp[i] = sub_res + 1;
            }
        }
    }
    return dp[V];
}
int main() {
```

```
int coins[] = {1, 5, 10};
int m = sizeof(coins)/sizeof(coins[0]);
int V = 12;

int result = minCoins(coins, m, V);
if (result == INT_MAX)
    cout << "No solution possible" << endl;
else
    cout << "Minimum coins required = " << result << endl;

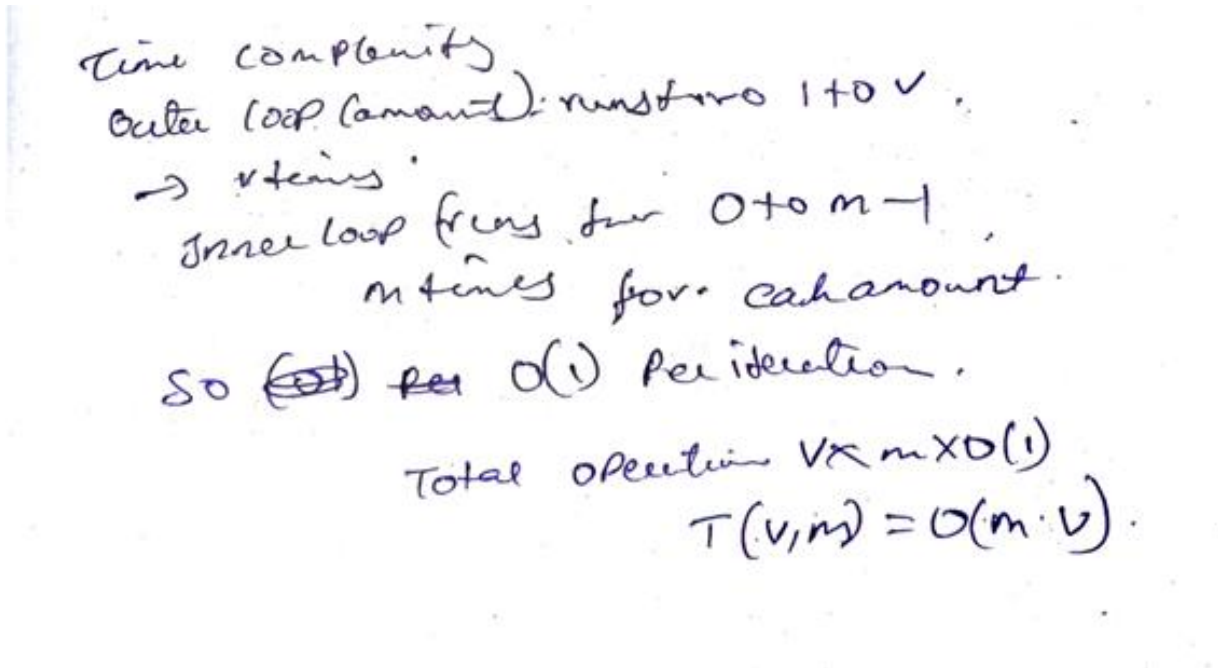
return 0;
}
```

3 OUTPUT:



C:\WINDOWS\system32\cmd. X
Minimum coins required = 3
C:\Users\YASWANTH\DAA>

4 TIME COMPLEXITY:



Time complexity
Outer loop (amount) runs from 1 to V.
→ coins.
Inner loop runs from 0 to m-1.
m times for each amount.
So ~~(m)~~ per $O(1)$ per iteration.
Total operation $V \times m \times O(1)$
 $T(V, m) = O(m \cdot V)$.

5 Conclusion:

The Making Change problem solved with Dynamic Programming ensures the minimum number of coins is found by building solutions for smaller amounts and combining them optimally.